AI Image Generator

1. Project Overview

Version 2.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Z-Image Turbo, Stable Diffusion XL and its best fine-tunes, FLUX.1, LoRAs and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and a live Admin panel.

1. Project Overview

1.1 Purpose

AI Image Generator is a self-hosted application that turns a plain-English prompt into an image using Stable Diffusion XL, its best community fine-tunes, or FLUX.1, running entirely on a local NVIDIA GPU. No image data leaves the machine and there is no per-image cost.

The project was built on 21 and 22 October 2026 and grew through eight increments:

1. A command-line script that runs Stable Diffusion XL on the GPU.
2. A local web interface so a prompt can be typed and the result viewed in a browser.
3. A prompt language that replaces sliders and dropdowns: size, ratio, quality, style, count and seed are all read from the prompt text.
4. Per-image telemetry (timings, VRAM, temperature, power, clocks) shown in the UI and stored in a SQLite database with a History tab.
5. A Real-ESRGAN upscaling stage so that "4k" or "8k" in the prompt produces a genuine 3840 or 7680 pixel image.
6. A model registry: six SDXL-family checkpoints, Z-Image Turbo and FLUX.1 schnell, automatic model choice by style, local checkpoints, LoRAs, prompt weighting and per-model samplers.
7. The Frazile AI studio interface: dark theme with pink-to-yellow accents, full-width layout, and an Upscale bar that appears after generation.
8. An Admin panel with live CPU, GPU, memory, storage, network and power telemetry, plus a Docker image for container deployment.

1.2 Key capabilities

- Text-to-image generation with eight built-in models: Z-Image Turbo (the default), SDXL base 1.0, Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo, Playground v2.5, Animagine XL 3.1 and FLUX.1 schnell, all in 16-bit precision (FLUX in 4-bit on 24 GB cards).
- Any SDXL checkpoint dropped into models/checkpoints/ and any SDXL LoRA in models/loras/ is usable immediately after a restart.
- Prompt-driven control of model, LoRAs, emphasis weighting, aspect ratio, exact resolution, orientation, quality, fidelity, art style, image count, seed, steps and guidance.
- Automatic model selection from style words (photorealistic goes to Juggernaut, anime to Animagine, painting to Playground, digital art to DreamShaper).
- Live interpretation line under the prompt box showing exactly what will be generated, with which model, before the button is pressed.
- Tiled Real-ESRGAN upscaling to 2K, 4K, 8K or any exact size, either from the prompt or afterwards from the Upscale bar, with automatic centre crop.
- Full telemetry per image: inference time, steps per second, peak VRAM, GPU temperature and power before and after, clocks, fan, performance state, driver and library versions.
- Persistent history in SQLite with search, aggregate statistics, and click-to-view of any past image with its full record, including later upscales.
- Admin panel: nine live tiles and six detail sections covering GPU, CPU, memory and storage, network, power, software and the app process, with optional 5-second auto-refresh.
- Command-line tools for batch generation with any model and LoRA, upscaling an existing image, querying the database and dumping system telemetry.

- A Dockerfile and docker-compose.yml for running the whole studio as a GPU container.

1.3 Target environment

The application was developed and verified on the following machine. Other NVIDIA GPUs with at least 12 GB of VRAM will work for the SDXL-family models; see the Requirements section for the full matrix.

Item	Value
GPU	NVIDIA GeForce RTX 3090, 24 GB VRAM, compute capability 8.6, 82 SMs
CPU / RAM	AMD Ryzen 5 3500 (6 cores), 16 GB
Driver	610.88
OS	Windows 11 Pro 10.0.26300
Python	3.12.0
CUDA runtime (via PyTorch)	12.6
Project folder	C:\Users\frazi\Documents\apps\ai-image-gen
Repository	https://github.com/Fzhacking/ai-image-gen (private)

1.4 Document map

Section	Contents
2. Requirements	Functional and non-functional requirements, hardware and software matrix, model assets
3. Architecture	Components, pipeline diagram, data flow, design decisions, concurrency
4. Workflows	Setup, generation, choosing models, LoRAs, upscaling, history, Admin panel, Docker, maintenance
5. Module reference	Every source file, its public functions and their contracts
6. Models	The model registry, automatic selection, samplers, prompt weighting, LoRAs, FLUX
7. Prompt language	Complete reference of words the parser understands
8. Database	Schema of generations.db, every column explained, example queries
9. Admin panel	Every tile and section of the live system panel and where its numbers come from
10. Performance	Measured timings and resource use on the RTX 3090 for every model
11. Deployment and Docker	Container image, compose file, volumes, GPU access, host requirements
12. Operations	Running, stopping, troubleshooting, known issues, backup, security
13. Change log and roadmap	What was built in which order, defects fixed, future work
Appendix	File tree, dependency list, command cheat sheet, API access, licences